

Python

Sommario

Python	1
Introduzione a Python.....	3
1. Cos'è Python?	3
2. Installazione di Python	3
3. Sintassi di Base.....	3
3.1 Indentazione	3
3.2 Commenti.....	4
3.3 Output.....	4
3.4 Input	4
3.5 Variabili e Tipi di Dati	4
3.6 Principali Tipi di Dati.....	5
3.7 Gestione date	5
4. Operatori.....	5
5. Strutture di Controllo	6
5.1 Condizioni (if, else).....	6
5.2 Cicli (for, while).....	6
6. Gestione errori.....	6
Stringhe in Python	8
1. Dichiarazione	8
2. Utilizzo come array e manipolazione	8
2.1 Array	8
2.2 Sottostringhe	8
2.3 principali metodi	9
3. Concatenazione	9
4. Segnaposto e formattazione	10
Liste, Tuple, Set, Dizionari in Python	11
1. Liste.....	11
1.1 Dichiarazione e manipolazione	11
1.2 Metodi.....	11
La lista è una classe	11
2. Tuple	12
2.1 Dichiarazione e manipolazione	13
2.2 Metodi.....	13
La tupla è una classe.....	13
3. Set	14

3.1 Dichiarazione e manipolazione	14
3.2 Metodi	14
4. Dizionari.....	16
4.1 Dichiarazione e manipolazione	16
4.2 Metodi	17
Python Package Index (PyPI)	19
1. PyPI (pip).....	19
2. Moduli nativi (Standard Library).....	20
3. Creazione Moduli	20
Array	22
1. Array	22
1.1 Dichiarazione	22
2. Numpy.....	23
2.1 Dichiarazione e metodi principali	23
Funzioni.....	25
1. Definizione e chiamata.....	25
2. Scope	26
3. Global.....	26
4. Passaggio parametri.....	26
Classi e Oggetti	28
1. Definizione e istanza.....	28
2. Ereditarietà.....	29
3. Polimorfismo	29
Gestione file	31
1. Lettura file.....	31
2. Scrittura file	31
3. Modulo randomuser.....	32
4. Modulo csv.....	32
4.1 Lettura file	32
5.2 Scrittura file.....	32
Tkinter	34
1. Esempio uso .pack().....	35
2. Esempio uso .grid().....	36
3. Esempio uso .place().....	37
4. Esempio uso MessageBox.....	37

Introduzione a Python

1. Cos'è Python?

Python è stato creato nel 1991 da Guido van Rossum. Negli anni ha subito molteplici cambiamenti e la versione attuale (ottobre 2024) è la 3.13. E' un linguaggio di programmazione ad alto livello, interpretato e versatile, molto utilizzato per sviluppo web, analisi dei dati, intelligenza artificiale e molto altro.

2. Installazione di Python

Si può scaricare Python dal sito ufficiale python.org. Selezionare la spunta Add Python 3.X to PATH all'inizio dell'installazione.

Eventualmente fare riferimento alla pagina dedicata sul sito ufficiale [Installazione](#)

Per verificare la corretta installazione basta scrivere in una shell di dos il comando
`python --version`

```
c:\> python --version
```

Esistono vari modi per utilizzare python da interprete interattivo, da IDLE o semplicemente digitando in una shell dos il comando `python script.py`

```
# Esempio di esecuzione script di python
python script.py
```

3. Sintassi di Base

Python è un linguaggio case-sensitive

3.1 Indentazione

In Python, utilizza l'indentazione e non parentesi graffe o altro per definire i blocchi di codice.

La Python Software Foundation consiglia di usare sempre 4 spazi per livello di indentazione; evitare l'uso dei caratteri di tabulazione; non mischiare mai l'uso di tabulazioni e spazi.

```
# Esempio di indentazione
if True:
    print("Questa riga è indentata.")
    print("Fa parte del blocco if.")
print("Questa riga non è indentata e non fa parte del blocco if")
```

3.2 Commenti

Si usa `#` oppure per commenti multilinea `""" commenti """`

```
# Questo è un commento su una sola riga
print("Ciao!") # Questo commento segue una riga di codice

"""
Questo è un commento multilinea.
Puoi utilizzarlo per spiegazioni più lunghe.
"""
```

3.3 Output

La funzione `print()` è utilizzato per visualizzare output sullo schermo.

```
# Esempio di stampa
print("Ciao, mondo!")
```

3.4 Input

La funzione `input()` restituisce sempre una stringa.

```
# Esempio di input
nome = input("Qual è il tuo nome?")
print("Ciao,", nome)
```

3.5 Variabili e Tipi di Dati

Le variabili vengono create al momento dell'assegnazione = non è necessario specificare il tipo di dato.

Volendo si può forzare il cast `int(3)`

```
# Esempi di variabili
nome = "Alice"
eta = 20
studente = True

print("Nome:", nome)
print("Età:", eta)
print("È uno studente?", studente)

# Se specifico tipo di dato "Casting"

x = str(3)    # x contiene la stringa '3' oppure "3"
y = int(3)    # y contiene il numero intero 3
z = float(3)  # z contiene il numero con la virgola 3.0

# Per sapere il tipo di dato
print(type(x))
print(type(y))
print(type(z))
```

3.6 Principali Tipi di Dati

- **Interi:** numeri senza decimali.
- **Float:** numeri in virgola mobile.
- **Stringhe:** sequenze di caratteri.
- **Booleani:** valori `True` o `False`.

3.7 Gestione date

In python le date non sono un tipo di dato vero e proprio, ma vengono implementate e gestite dal modulo `datetime`.

```
from datetime import datetime

oggi = datetime.now()
print(oggi)

orario = oggi.strftime("%H:%M:%S") #converte datetime in stringa
print("Ora attuale:", orario)
print("Ora: ", oggi.hour)
print("Minuti: ", oggi.minute)
print("Secondi: ", oggi.second)
data = oggi.strftime("%d-%m-%Y") #converte datetime in stringa
print("Data attuale:", data)
print("Anno: ", oggi.year)
print("Mese: ", oggi.month)
print("Giorno: ", oggi.day)

dataInizio = datetime.strptime("01-01-1900", "%d-%m-%Y") #converte stringa in
datetime
dataFine = datetime(2024, 12, 31) #crea una data (anno, mese, giorno)
diff = dataFine - dataInizio
print(f"Dal 01-01-1900 al 31.12-2024 sono passati: {diff.days} giorni")
```

4. Operatori

Python supporta diversi tipi di operatori, inclusi operatori aritmetici, di confronto e logici.

```
# Esempi di operatori
a = 10
b = 5

somma = a + b
print("Somma:", somma)

maggiore = a > b
print("A è maggiore di B?", maggiore)
```

5. Strutture di Controllo

5.1 Condizioni (if, else)

Le istruzioni condizionali permettono di eseguire codice in base a condizioni.

```
# Esempio di condizione
numero = 7
if numero % 2 == 0:
    print("Il numero è pari.")
else:
    print("Il numero è dispari.")
```

5.2 Cicli (for, while)

I cicli consentono di ripetere un blocco di codice.

```
# Esempio di ciclo for
for i in range(5):
    print("Iterazione:", i)
# Esempio di ciclo while
conto = 0
while conto < 5:
    print("Conto:", conto)
    conto += 1
```

6. Gestione errori

La gestione degli errori è implementata con

- `try:` istruzioni da controllare
- `except:` gestione degli errori
- `else:` istruzioni se non ci sono errori
- `finally:` istruzioni sempre eseguite (con o senza errori)

`else` e `finally` opzionali

Vedere https://www.w3schools.com/python/python_ref_exceptions.asp

```
x=3
try:
    print(f"ok - non ci sono errori x={x}")
except:
    print("except - istruzioni da eseguire se c'è un errore")
else:
    print("else - istruzioni da eseguire se non ci sono errori")
finally:
    print("finally - istruzioni sempre eseguite")

#gestione del tipo di eccezione
```

```
try:
    n = int("cinque")
except ValueError:
    print("Numero non valido")

try:
    n = 10/ 0
except ZeroDivisionError:
    print("Divisione per zero")
except ValueError:
    print("Numero non valido")
except NameError:
    print("Nome non valido")
```

Stringhe in Python

1. Dichiarazione

Le stringhe in Python sono delimitate indifferentemente da singoli apici o virgolette.

E' anche possibile dichiarare stringhe *multiline* utilizzando tre apici o virgolette.

Ai caratteri speciali come `\`, `"`, `'`, `\t` (tabulazione), `\n` (a capo), va anteposto un backslash `\\`, `\"`, `\'`, `\t`, `\n`

```
# Definizione di stringhe
s1 = 'Ciao Mondo!'
s2 = "Hello world"
s3 = "Il mio nome è 'Pippo'"
s4 = 'Questo è un Corso di "Python"'

print(s1)
print(s2)
print(s3)
print(s4)
print("") #riga vuota

# Stringhe multiline
s5 = """Questa è una stringa
disposta su più righe"""
s6 = '''Anche questa è una stringa
disposta su più righe'''

print(s5)
print("") #riga vuota
print(s6)
```

2. Utilizzo come array e manipolazione

2.1 Array

Le stringhe sono rappresentate come array di byte (caratteri) dove il primo carattere ha indice zero.

```
# Stringhe come Array
s = "Salve Mondo"
print("Stampo il settimo carattere:", s[6]) # Stampa 'M'

# Altro esempio
for x in 'Hello World':
    print(x)
```

2.2 Sottostringhe

Per ottenere sottostringhe occorre inserire tra parentesi quadre la posizione iniziale e finale

```
# Stringa
s = "Salve Mondo"
# Sottostringa
s1 = s[2:5]
s2 = s[:5]
s3 = s[6:]

print(s) # "Salve Mondo"
print(s1) # "lve"
print(s2) # "Salve"
print(s3) # "Mondo"
```

2.3 principali metodi

Le stringhe espongono molteplici metodi tra cui i principali sono:

- `upper()` e `lower()` convertono in maiuscolo e minuscolo
- `strip()` elimina eventuali spazi prima e dopo
- `replace()` sostituisce una sottostringa con un'altra
- `split()` separa le sottostringhe in base ad un carattere separatore

Si veda <https://docs.python.org/3/library/stdtypes.html#string-methods>

```
# upper() lower()
s = "Salve Mondo"
print(s.upper()) #"SALVE MONDO"
print(s.lower()) #"salve mondo"

# strip()
s = " Salve Mondo "
print(s.strip()) #"Salve Mondo"

# replace()
s = "Salve Mondo"
s1 = s.replace("nd", "ND")
print(s1) #"Salve MoNDo"

# split()
s = "Salve;Mondo"
print(s.split(";")) #['Salve', 'Mondo']
```

3. Concatenazione

Utilizzare l'operatore +

N.B. non è possibile concatenare stringhe e numeri (genera un errore quando il programma viene compilato), ma occorre fare il cast

```
# Concatenazione
s1 = "Unire"
```

```

s2 = "stringhe"
s3 = "in"
s4 = "Python"
sp = "_" # separatore

s = s1 + sp + s2 + sp + s3 + sp + s4

print(s) #"Unire_stringhe_in_Python"

# Concatenazione ERRATA produce un errore nel compilatore
s = "La mia età è: "
eta = 18
print(s + eta) #ERRORE

# CORRETTA
print(s + str(eta)) #"La mia età è: 18"

```

4. Segnaposto e formattazione

Per inserire stringhe o altri tipi di dato/operazione matematica in una determinata posizione di una stringa occorre anteporre `f` alla stringa di partenza ed inserire i segnaposti dentro `{...}` (Viene eseguito automaticamente il cast)

```

# Segnaposto
eta = 18
s = f"Oggi compio {eta} anni"

print(s) #"Oggi compio 18 anni"

a = 4
b = 6
somma = f"La somma di {a} e {b} vale {a+b}"
divisione = f"La divisone di {a} e {b} vale {a/b:3.4f}"

print(somma) #"La somma di 4 e 6 vale 10"
print(divisione) #"La divisione di 4 e 6 vale 0.6667"

```

Liste, Tuple, Set, Dizionari in Python

In Python esistono 4 tipi di **collection** di dati, mentre gli array non hanno un supporto nativo e solitamente vengono utilizzate le liste che offrono le stesse funzionalità.

Se si volessero utilizzare gli array occorre importare apposite librerie come ad esempio [array](#) o [numpy](#).

1. Liste

In Python le liste sono supportate nativamente e permettono di rappresentare una sequenza **mutabile** di elementi indicizzati che possono cambiare, essere aggiunti ed eliminati, ammettere duplicati, non essere dello stesso tipo.

Vengono dichiarate mediante `[1, 2, 3]` oppure `lst=list((1, 2, 3))`.

1.1 Dichiarazione e manipolazione

```
lstFrutta = ["mela", "banana", "susina", "caco", "kiwi"]
print("La mia frutta preferita: ", lstFrutta)

for x in lstFrutta:
    print(x)
print("Altro metodo")
for i in range(len(lstFrutta)):
    print(lstFrutta[i])

lstMista = ["ciao", 77, False]
print("Lista non omogenea ", lstMista)

lstFrutta[1] = "prugna" # indice da 0 a n-1
print(lstFrutta)

print(lstFrutta[2:4]) #['susina', 'caco']
print(lstFrutta[:3]) #['mela', 'prugna', 'susina']
print(lstFrutta[2:]) #['susina', 'caco', 'kiwi']

if "susina" in lstFrutta:
    print("trovato")
```

1.2 Metodi

La lista è una classe

```
lstFrutta = list(("mela", "banana", "susina", "caco", "kiwi"))
print("lstFrutta ", lstFrutta)

print(type(lstFrutta)) #<class 'list'>

print("Numero elementi ", len(lstFrutta))

lstFrutta.append("kiwi") #aggiunge al fondo
lstCopia.append("kiwi") #duplicato
```

```

print("append ", lstFrutta)

lstFrutta.insert(3,"pompelmo") #aggiunge in posizione specificata (base 0)
print("insert ", lstFrutta)

lstVerdura = ["patata", "carota", "cavolo"]
print("lstVerdura ",lstVerdura)

lstUnisci = lstVerdura + lstFrutta #unisce 2 o più liste
print("lstUnisci", lstUnisci)

lstFrutta.extend(lstVerdura) #unisce una lista al fondo dell'altra
print("extend ", lstFrutta)

lstFrutta.remove("caco") #rimuove un elemento specificato
print("remove ",lstFrutta)

lstFrutta.pop(2),
print("pop ", lstFrutta) #rimuove un elemento a una specifica posizione

del(lstVerdura[1]) #rimuove un elemento a una specifica posizione
print("del ", lstVerdura)

lstVerdura.clear() #svuota la lista
print("clear ", lstVerdura)

lstFrutta.sort() #ordinamento crescente
print("Lista ordinata crescente ", lstFrutta)

lstFrutta.sort(reverse=True) #ordinamento decrescente
print("Lista ordinata decrescente", lstFrutta)

lstCopia = lstFrutta.copy() #copia una lista
print("lstCopia",lstCopia)

n = lstCopia.count("kiwi") #conta quante volte è presente un elemento
print(f"kiwi presente {n} volte")

pos = lstCopia.index("kiwi") #restituisce la prima occorrenza di un elemento
print(f"kiwi trovato la prima volta in posizione {pos}")

```

2. Tuple

Le tuple sono supportate nativamente e permettono di rappresentare una sequenza **immutabile** di elementi che non si possono modificare, aggiungere od eliminare, ma sono indicizzate e ammettono duplicati e tipi misti.

Vengono dichiarate mediante (1, 2, 3) oppure `tpl=tuple((1, 2, 3)).`

2.1 Dichiarazione e manipolazione

Creare una tupla viene definito **packing** ed è possibile effettuare l'operazione inversa **unpacking** ossia assegnare ogni elemento ad una variabile.

Posso anche convertire una tupla in lista, unire 2 tuple con l'operatore + e moltiplicarne gli elementi.

```
tplFrutta = ("mela", "banana", "susina", "caco", "kiwi") #packing
print("La mia frutta preferita: ", tplFrutta)

for x in tplFrutta:
    print(x)
print("Altro metodo")
for i in range(len(tplFrutta)):
    print(tplFrutta[i])

tplMista = ["ciao", 77, False]
print("Tupla non omogenea ", tplMista)

try:
    tplFrutta[1] = "prugna"
    print(tplFrutta)
except:
    print("Errore non permesso")

print(tplFrutta[2:4]) #['susina', 'caco']
print(tplFrutta[:3]) #['mela', 'prugna', 'susina']
print(tplFrutta[2:]) #['susina', 'caco', 'kiwi']

if "susina" in tplFrutta:
    print("trovato")

(a,b,c,d,e) = tplFrutta #unpacking
print ("unpacking ", a, b, c, d, e)

lstFrutta = list(tplFrutta)
print("conversione in lista ", lstFrutta)

tplUnisci = tplFrutta + tplFrutta #unisce 2 o più tuple
print("unisci ", tplUnisci)

tplMoltiplica = tplFrutta * 3 #triplica gli elementi
print("moltiplica ", tplMoltiplica)
```

2.2 Metodi

La tupla è una classe

```
tplFrutta = tuple(("mela", "banana", "susina", "caco", "kiwi"))
print("tplFrutta ", tplFrutta)

print(type(tplFrutta)) #<class 'tuple'>
```

```

print("Numero elementi ", len(tplFrutta))

tplUnisci = tplFrutta + tplFrutta #unisce 2 o più tuple
print("tplUnisci", tplUnisci)

n = tplUnisci.count("kiwi") #conta quante volte è presente un elemento
print(f"kiwi preente {n} volte")

pos = tplFrutta.index("kiwi") #restituisce la prima occorrenza di un elemento
print(f"kiwi trovato la prima volta in posizione {pos}")

```

3. Set

I set sono supportati nativamente e permettono di rappresentare una sequenza di elementi che non si possono modificare, ma aggiungere ed eliminare. Non sono indicizzati, non hanno un ordine (possono venir visualizzati in ordine diverso) e non ammettono duplicati. Offrono funzionalità insiemistiche (unione, intersezione, differenza). Vengono dichiarati mediante {1, 2, 3}.

3.1 Dichiarazione e manipolazione

Esiste la possibilità di creare set con elementi di più tipi, ma True e 1 e false e 0 sono visti come lo stesso elemento e quindi uno dei 2 viene eliminato.

```

stFrutta = {"mela", "banana", "susina", "caco", "kiwi"} #packing
print("La mia frutta preferita: ", stFrutta)

for x in stFrutta:
    print(x)
stMista = {"ciao", 0, True, "mondo", 1, False}
print("Set non omogeneo ", stMista)

if "susina" in stFrutta:
    print("trovato")

(a,b,c,d,e) = stFrutta #unpacking
print ("unpacking ", a, b, c, d, e)

lstFrutta = list(stFrutta)
print("conversione in lista ", lstFrutta)

```

3.2 Metodi

Il set è una classe

```

stFrutta = set(("mela", "banana", "susina", "caco", "kiwi"))
print("stFrutta ", stFrutta)
print(type(stFrutta)) #<class set>

stFrutta1 = {"pompelmo", "fragola", "banana", "caco"}
print("stFrutta1 ", stFrutta1)

```

```

stFrutta2 = stFrutta1.copy()
print("copy ", stFrutta2)

print("Numero elementi ", len(stFrutta))

stFrutta.add("arancia") #aggiunge un elemento
print("append ", stFrutta)

stFrutta2.update(stFrutta) #unisce i 2 set
print("update ", stFrutta2)

lstVerdura = ["patata", "carota", "cavolo"]
print("lstVerdura ",lstVerdura)

stFrutta2.update(lstVerdura) #unisce un set ad una lista o tupla
print("update ", stFrutta2)

if "susina" in stFrutta2:
    stFrutta2.remove("susina") #rimuove un elemento specificato
                                #se non lo trova genera errore
    print("remove ",stFrutta2)

stFrutta2.discard("susina") #rimuove un elemento specificato
                            #se non lo trova NON genera errore
print("discard ",stFrutta2)

stFrutta3 = stFrutta.union(stFrutta1) #unisce 2 o più set o liste/tuple
print("union ",stFrutta3)

stFrutta4 = stFrutta.intersection(stFrutta1) #intersezione di 2 set
                                                #(elementi presenti in entrambi)
print("intersection ",stFrutta4)

stFrutta5 = stFrutta.difference(stFrutta1) #differenza tra 2 set
                                                #(elementi presenti nel primo
                                                # e non nel secondo)
print("difference ",stFrutta5)

stFrutta2.pop(),
print("pop ", stFrutta2) #rimuove un elemento in posizione casuale,
                        #perchè set non ordinato

stFrutta2.clear() #svuota il set
print("clear ", stFrutta2)

del(stFrutta) #elimina set
print("del ", stFrutta)

```

4. Dizionari

I dizionari sono supportati nativamente e permettono di rappresentare una collection di elementi modificabili, ordinati (dalla versione 3.7) e non ammettono duplicati. Gli elementi sono rappresentati nella notazione chiave-valore.

```
dctAuto = {  
    "marca": "fiat",  
    "modello": "Doblo",  
    "anno": 2013  
}
```

4.1 Dichiarazione e manipolazione

Per item si intende la singola coppia chiave-valore, si accede agli item mediante il valore della chiave, si possono avere più dizionari annidati (entità) che permettono di rappresentare più entità non necessariamente dello stesso tipo o con numero di item o item diversi (ad esempio un insieme di auto (entità) con marche, modelli diversi e caratteristiche (item) diversi

```
#dichiarazione Dizionario  
dctAuto = {  
    "auto1" : {  
        "marca": "fiat",  
        "modello": "Doblo",  
        "anno": 2013  
    },  
    "auto2" : {  
        "marca": "fiat",  
        "modello": "Punto",  
        "anno": 2010  
    },  
    "auto3" : {  
        "marca": "fiat",  
        "modello": "Punto",  
        "cambio": "Automatico"  
    }  
}  
  
print("Dizionario ",dctAuto) #visualizza il dizionario  
print("Entità ",dctAuto["auto1"]) #visualizza entità  
print("Item",dctAuto["auto2"]["marca"]) #visualizza item  
  
dctAuto["auto1"]["marca"] = "Ferrari" #modifica un item  
dctAuto["auto2"] = {"marca" : "Audi", "colore" : "Blu Patata"} #modifica  
                                                                #entità  
  
print("Dizionario ",dctAuto) #visualizza il dizionario  
  
for x in dctAuto: #stampa ogni entità senza gli item  
    print(x)  
    for y in dctAuto: #stampa gli item di una entità  
        print(dctAuto[y])  
  
print("lunghezza ",len(dctAuto)) #visualizza numero entità
```



```

dctAuto["auto3"]["porte"] = 5 #aggiunge un item
print("Entità ",dctAuto["auto3"]) #visualizza entità

print("lunghezza ",len(dctAuto)) #visualizza numero entità

del(dctAuto["auto1"]["modello"]) #rimuove un item
del(dctAuto["auto2"]) #rimuove un'entità
print(dctAuto)

del(dctAuto) #cancella il dizionario
try:
    print(dctAuto) # genera errore
except:
    print("dctAuto non definito")

```

4.2 Metodi

Il dizionario è una classe

```

#dichiarazione Dizionario
dctAuto = {
    "auto1" : {
        "marca": "fiat",
        "modello": "Doblo",
        "anno": 2013
    },
    "auto2" : {
        "marca": "fiat",
        "modello": "Punto",
        "anno": 2010
    },
    "auto3" : {
        "marca": "fiat",
        "modello": "Punto",
        "cambio": "Automatico"
    }
}
print("dctAuto ",dctAuto) #visualizza il dizionario

dctAuto1 = dctAuto.copy() #copia un dizionario
print("dctAuto1 ",dctAuto1)

modello = dctAuto["auto1"].get("modello") #restituisce il valore di una chiave
                                           #none se non lo trova
print("modello ",modello)

auto = dctAuto.get("auto2") #restituisce il valore di una entità
                             #none se non lo trova
print("auto ",modello)

entita = dctAuto.items() #restituisce il PUNTATORE alle entità (chiave-valore)
                         #di un dizionario
print("entità ",entita)

```

```

item = dctAuto["auto3"].items()
print("item ",item) #restituisce il PUNTATORE agli item (chiave-valore)
                    #di un item

chiavi = dctAuto.keys() #restituisce il PUNTATORE alle chiavi delle entità
                    #di un dizionario
print("chiavi ",chiavi)

chiavi = dctAuto["auto3"].keys()
print("chiavi ",chiavi) #restituisce il PUNTATORE alle chiavi
                    #di una entità

valori = dctAuto.values() #restituisce il PUNTATORE ai valori delle entità
                    #di un dizionario
print("valori ",valori)

valori = dctAuto["auto1"].values() #restituisce il PUNTATORE ai valori
                    #delle chiavi di una entità
print("valori ",valori)

nuovaAuto = {"auto4" : {
    "marca": "Ford",
    "modello": "Escort",
    "cambio": "manuale"
}}
dctAuto1.update(nuovaAuto) #inserisce una nuova entità/item
print("dctAuto1 ",dctAuto1)

risultato = dctAuto1.pop("auto1","non trovata") #elimina l'entità/item
                                                #restituisce l'item se trovato
                                                #o il valore default, errore
                                                #se default non specificato

print("risultato ",risultato)
print("dctAuto1 ",dctAuto1)

```

Python Package Index (PyPI)

1. PyPI (pip)

E' un repository di software per la programmazione in Python <https://pypi.org/> dove è possibile cercare, scaricare e caricare librerie e moduli. Se non è presente si può installare da <https://pypi.org/project/pip/>

Controllare la versione

```
c:\> pip --version
```

Eseguire l'upgrade del repository

```
c:\> python - m pip install --upgrade pip
```

Eseguire l'upgrade di un pacchetto

```
c:\> pip install --upgrade camelcase
```

Per installare un pacchetto

```
C:\>pip install camelcase
Collecting camelcase
Downloading camelcase-0.2.tar.gz (1.3 kB)
...
Installing build dependencies ... done
Installing collected packages: camelcase
Successfully installed camelcase-0.2
```

Per utilizzare un pacchetto

```
import camelcase
c = camelcase.CamelCase()
txt = "hello world"
print(c.hump(txt)) #Hello World
```

Per disinstallare un pacchetto

```
C:\>pip uninstall camelcase
Found existing installation: camelcase 0.2
Uninstalling camelcase-0.2:
  Would remove:
    c:\users\admin\appdata\...\python\...\camelcase-0.2.dist-info\*
    c:\users\admin\appdata\...\python\...\camelcase\*
Proceed (Y/n)?
```

Per vedere i pacchetti installati

```
C:\>pip list
Package      Version
-----
anyio        4.1.0
...
camelcase    0.2
...
```

Per controllare i pacchetti installati e le loro dipendenze

```
C:\>pip check
No broken requirements found
```

Per visualizzare i comandi e la sintassi (senza COMANDO help generale)

```
C:\>pip COMANDO --help
Usage:
pip <command> [options]
... ..
```

2. Moduli nativi (Standard Library)

La Standard Library <https://docs.python.org/it/3.13/library/index.html> comprende una molteplice raccolta di moduli in grado di fornire funzionalità avanzate nello sviluppo di applicazioni. (random, math, datetime,)

Analogamente ai pacchetti PyPI occorre fare la `import`, ma essendo nativi non occorre utilizzare `pip install`

```
import random as rnd
import math
import datetime as dt

for numero in range(10):
    val = rnd.randint(1, 50)
    print(val)

print(round(math.sqrt(15),2)) #round() già nativa in python

oggi = dt.datetime.today()
print(oggi)
oggi = dt.datetime.today().strftime('%d-%m-%y')
print(oggi)
```

3. Creazione Moduli

È sufficiente inserire il codice da riutilizzare in un file con estensione `.py` ed importare il modulo creato.

libListe.py

```
import random as rnd

def creaListaInt(n:int, lst:list): #posso definire il tipo di parametro
    for numero in range(n):
        lst.append(rnd.randint(1, 100))

def caricaLista(n:int, lst:list):
```

```

for numero in range(n):
    x = input(f"Inserire elemento {numero}: ")
    lst.append(x)

def stampaLista(n:int, lst:list):
    for numero in range(n):
        print(f"Elemento {numero}: {lst[numero]}")

```

programma.py

```

#ATTENZIONE in VisualStudioCode aprire la cartella
#dove si trovano il modulo e il programma
#viene creata una cartella __pycache__
#che contiene il bytecode del modulo compilato
import libListe

a=[] #lista vuota

libListe.creaListaInt(10,a)
libListe.stampaLista(10,a)

b=[]
libListe.caricaLista(3,b)
libListe.stampaLista(3,b)

```

Array

Gli array in Python non sono supportati nativamente ed occorre appoggiarsi a librerie esterne come ad esempio [array](#) o [numpy](#). Al loro posto si possono utilizzare le liste che offrono le stesse funzionalità.

1. Array

Sono molto simili alle liste (mutabili, indicizzati, ammettono duplicati, ma gli item devono essere dello stesso tipo) ed espongono lo stesso set di metodi. È consigliabile utilizzarli quando si eseguono operazioni matematiche su insiemi omogenei.

Metodo	Descrizione
<code>append()</code>	Aggiunge un elemento alla fine dell'array.
<code>clear()</code>	Rimuove tutti gli elementi dell'array.
<code>copy()</code>	Restituisce una copia dell'array.
<code>count()</code>	Restituisce il numero di occorrenze di un valore specificato nell'array.
<code>extend()</code>	Aggiunge gli elementi di un array (o di qualsiasi iterabile) alla fine dell'array corrente.
<code>index()</code>	Restituisce l'indice della prima occorrenza di un valore specificato.
<code>insert()</code>	Aggiunge un elemento in una posizione specificata dell'array.
<code>pop()</code>	Rimuove e restituisce l'elemento nella posizione specificata (o l'ultimo elemento se non specificato).
<code>remove()</code>	Rimuove la prima occorrenza di un valore specificato nell'array.
<code>reverse()</code>	Inverte l'ordine degli elementi nell'array.
<code>sort()</code>	Ordina gli elementi dell'array in ordine crescente (per default).

1.1 Dichiarazione

Tipo	Tipo in C	Tipo in Python	Bytes
'b'	signed char	int	1
'B'	unsigned char	int	1
'h'	signed short	int	2
'H'	unsigned short	int	2
'i'	signed int	int	2
'I'	unsigned int	int	2
'l'	signed long	int	4
'L'	unsigned long	int	4
'q'	signed long long	int	8
'Q'	unsigned long long	int	8
'f'	float	float	4
'd'	double	float	8

La dichiarazione di un array avviene tramite il costruttore `array.array('i', [1,2,3])`

```
import random
import array

#costruttore
a = array.array('i') #array vuoto
b = array.array('u',['a','b','c']) #array con dati assegnati

print(type(a))

for i in range(10):
    a.append(random.randint(1, 100))
    #oppure
    #a[i]=(random.randint(1, 100))
print(a)
```

```
for c in b:
    print(c)
```

Altre modalità di dichiarazione:

```
import array as ar

a = ar.array('i', [1,2,3,4])
print(a)
```

```
from array import *

a = array('i', [1,2,3,4])
print(a)
```

Per accesso, modifica, aggiunta, eliminazione, ricerca e stampa fare riferimento alla dispensa sulle liste.

2. Numpy

Installare la libreria `numpy` utilizzando il package manager di Python `pip`

2.1 Dichiarazione e metodi principali

```
import numpy as np

a = np.array([17, 2, 43, 55 ,17 , 5])
b=np.array(['a','b','c'])
print(type(a))
print(a)
print(b)

b[2]='z'
for c in b:
    print(c)

print(np.sort(a))
print(a) #il metodo sort non modifica l'array

d=np.concatenate((a,b))
print(d)

x = np.where(a == 17)
print(x)

x = a.view() #puntatore ad a
a[0] = 42

print(a)
print(x)

y = a.copy() #copia di a
```

```
a[1] = 77
```

```
print(a)
```

```
print(y)
```


Funzioni

Le funzioni sono definite mediante la keyword `def`. Analogamente agli altri linguaggi possono ricevere nessuno o più parametri e restituire nessuno o più valori che possono essere in numero ed ordine diversi dalla firma della funzione.

1. Definizione e chiamata

```
def saluta(): #senza parametri non restituisce valori
    print("Salve Mondo")
    return #non obbligatorio

saluta()

def nome_cognome(n,c): #con parametri restituisce valore
    return n + " " + c

print(nome_cognome("Pippo","DePippis"))

def somma(*n): #numero variabile di parametri
    s=0
    for x in n:
        s=s+x
        print(x)
    return s

ris = somma(10,20,30,40)
print (ris)
ris = somma(1,2,3)
print (ris)

def messaggio(m = "Salve mondo"): #valore di default
    print(m)

messaggio()
messaggio("Ciao Mondo")

def vuota(): #pass permette di dichiarare una funzione vuota senza ricevere errore
    pass

def swap(x,y):
    return y, x

a=5
b=6

print(f"Prima a={a} - b={b}")
aaa,bbb = swap(a,b)
print(f"Dopo a={aaa} - b={bbb}")

def contoRovescia(n=10): #ricorsione
    if n>0:
        print(n)
```

```

    contoRovescia(n-1)
else:
    print("Fine")

contoRovescia()

```

2. Scope

Una variabile è visibile soltanto all'interno dell'area in cui è stata dichiarata `scope`. Le variabili dichiarate localmente ad una funzione non sono accessibili dall'esterno, mentre le variabili dichiarate al di fuori sono accessibili, in sola lettura, all'interno della funzione.

```

x = 300 #(globale) dichiarata nel corpo principale del programma
        #(nameSpace globale) sempre visibile
def f():
    x=x+1 #genera errore accessibile solo in lettura
    s = "Dentro la funzione " #nameSpace locale
    print(s,x) # x accessibile perchè globale

f()

print(s) #genera errore perchè dichiarata (locale) in f()
print(f"Fuori dalla funzione {x}")

```

3. Global

La keyword `global` permette di dichiarare una variabile con scope globale all'interno di una funzione e di rendere modificabile una variabile globale dentro una funzione.

```

def f():
    global x #x diventa globale
    x = 10

f()

x = x + 1
print(f"f = {x}")

```

4. Passaggio parametri

I parametri sono passati in modi diversi a seconda del tipo di parametro (mutabile o immutabile) **call-by-object**

- **valore**: se il parametro è immutabile (numero, stringa o tupla)
- **referenza**: se il parametro è mutabile (lista, set, dizionario)

```

def valore(x):
    x = x * 2
    print(f"Dentro valore() {x}")

def referenza(l):

```

```
l.append("D")
print("Dentro referenza() ", l)

lst = ["a", "b", "c"]
n = 6

print(f"n prima di valore() {n}")
valore(n)
print(f"n dopo valore() {n}")

print("lst prima di referenza() ",lst)
referenza(lst)
print("lst dopo referenza() ",lst)
```

Classi e Oggetti

Python è un linguaggio di programmazione orientato agli oggetti e quasi tutte le sue componenti sono oggetti.

1. Definizione e istanza

```
class Cane:
    #proprietà posso definirle anche nel costruttore
    colore: str
    #
    def __init__(self, n, e, c = None): #costruttore
        self.nome = n
        self.eta = e
        self.colore = c
    def __str__(self):
        return f"{self.nome} - {self.eta} - {self.colore}"
    def saluta(self): #metodo
        print("Bauuu sono " + self.nome)

cane = Cane("Pippo", 3) #istanza classe (oggetto)
cane.saluta() #richiamo metodo
cane.colore = "Marrone" #imposto proprietà
print(cane.eta)
print(cane.colore)
print(cane)

del cane.eta #cancella una property
try:
    print(cane.eta) #errore
except AttributeError:
    print("Proprietà inesistente")

del cane #cancella un oggetto
try:
    print(cane) #errore
except NameError:
    print("Oggetto inesistente")
```

`__init__()` costruttore (non obbligatorio) sempre eseguito quando la classe viene istanziata. Permette di definire le proprietà e le impostazioni iniziali della classe.

`self` parametro aggiuntivo ai metodi, si riferisce all'istanza corrente e permette di accedere alle proprietà. Se voglio definire le proprietà nel costruttore devo anteporlo.

`__str__` metodo opzionale che se presente restituisce una stringa per rappresentare la classe, Se non presente restituisce la rappresentazione dell'oggetto

2. Ereditarietà

È supportata anche l'ereditarietà multipla.

```
class Persona:
    def __init__(self, nome, cognome):
        self.nome = nome
        self.cognome = cognome
    def nomeCompleto(self):
        s = self.nome + " " + self.cognome
        return s

persona = Persona("Paolino", "Paperino")
print(persona.nomeCompleto())

class StudenteBase(Persona): #classe ereditata
    pass #in questo caso eredita senza aggiungere funzionalità

s = StudenteBase("Topolino", "Mouse")
print(s.nomeCompleto())

class Studente(Persona): #classe ereditata
    def __init__(self, nome, cognome, spec): #override
        self.spec = spec
        #se voglio continuare ad usare il costruttore della classe padre
        super().__init__(nome, cognome)
    def nomeCompleto(self): #override
        print(self.nome, self.cognome, self.spec)
        #oppure
        print("Con Super: ", super().nomeCompleto(), self.spec)
    def specializzazione(self):
        print(self.spec)

stud = Studente("Paperone", "DePaperoni", "Economia")

stud.nomeCompleto()
stud.specializzazione()
```

3. Polimorfismo

```
class Veicolo:
    def __init__(self, marca, modello):
        self.marca = marca
        self.modello = modello
    def velocita(self):
        print("km/h")

class Auto(Veicolo):
    pass
class Aereo(Veicolo):
    def velocita(self):
        print("Mach")
```

```
auto = Auto("Fiat", "Doblo")
aereo = Aereo("Boeing", "747")

for x in (auto, aereo):
    print(x.marca, x.modello) #mantengono la stessa interfaccia
    x.velocita()
```

Gestione file

La gestione di un file è del tutto simile agli altri linguaggi.

```
f=open(), f.read(), f.readline(), f.readlines(), f.write(), f.close()
```

L'opzione `newline=''` della `open()` serve a gestire l' "a capo" che in Windows è `\r\n` mentre in Unix/Linux è `\n` ed evita che vengano inserite righe vuote durante la scrittura.

```
libri.txt
```

```
A1C;Il C spiegato a tutti;23.77
A2A;C Sharp contro Python;12.45
H77;Dal bit al byte;33.78
```

1. Lettura file

```
nomeFile = "./libri.txt"

#apro il file in lettura
f = open(nomeFile, 'r', newline='')
#leggo il file
print(f.read()) #legge tutto il file
print(f.read(5)) #legge i primi 5 caratteri
print(f.readline()) #legge una riga alla volta
print(f.readlines()) #carica tutte le linee in una lista di stringhe
for riga in f: #scorre il file fino alla fine
    print(riga.split(';'))
#potrei usare il while
while True:
    riga = f.readline()
    if riga:
        print(riga.split(';'))
    else:
        print("fine")
        break
#chiudo il file
```

2. Scrittura file

```
nomeFile = "./libri.txt"

#apro il file
# 'a' append aggiunge alla fine
# 'w' rewrite se il file non esiste lo crea, altrimenti lo cancella e riscrive
# 'x' crea il file errore se esiste
f = open(nomeFile, 'a', newline='') # 'w' 'x'
#scrivo una riga
f.write("\n nuova riga")
#chiudo il file
f.close()
```

3. Modulo randomuser

Libreria per generare utenti casuali <https://pypi.org/project/randomuser/>

```
C:\>pip install randomuser
```

```
import randomuser
from randomuser import RandomUser as ru

# Recupera 5 utenti casuali
users = ru.generate_users(5)
# visualizzo gli utenti
for user in users:
    print("Cognome:", user.get_last_name())
    print("Nome:", user.get_first_name())
    print("Email:", user.get_email())
    print("Data di nascita:", user.get_dob())
    print("Cellulare:", user.get_cell())
    print("Foto:", user.get_picture())
    print("\n")
```

4. Modulo csv

Libreria per la lettura/scrittura di file csv <https://docs.python.org/it/3.13/library/csv.html>

4.1 Lettura file

```
import csv

nomeFile = "./libri.txt"
f = open(nomeFile, mode='r', newline='')
#oggetto reader
csv_reader = csv.reader(f, delimiter=';')
#scorro le righe ed estraggo i campi
for row in csv_reader:
    print(row)
    for c in row:
        print(c)
f.close()
```

5.2 Scrittura file

```
import csv

nomeFile = "./libri.txt"
# Dati da scrivere
dati = [
    ["A1C", "Il C spiegato a tutti", 23.77],
    ["A2A", "C Sharp contro Python", 12.45],
    ["H77", "Dal bit al byte", 33.78]
]
f = open(nomeFile, mode='w', newline='')
```



```
#oggetto writer
csv_writer = csv.writer(f, delimiter=';')
#Scrivo le righe nel file CSV
for row in dati:
    csv_writer.writerow(row)
f.close()
```

Tkinter

Tkinter è il modulo standard di Python per la creazione di interfacce grafiche (GUI). Mette a disposizione oltre ai principali controlli (label, textbox, button) tre metodi `pack()`, `place()` e `grid()` per posizionarli all'interno del form. Si sconsiglia l'uso contemporaneo dei tre metodi.

```
import tkinter as tk
from tkinter import messagebox
from tkinter import ttk # Per i widget avanzati combobox

# Creazione form
frm = tk.Tk()
frm.title("Esempio tkinter")
frm.geometry("400x300")

# Creo label
lblNome = tk.Label(frm, text="Come ti chiami?", bg="white", fg="blue")
lblNome.place(x=3,y=20)

#creo textBox
txtNome = tk.Entry(frm)
txtNome.place(x=100,y=20)
txtNome.insert(0,"prova") #scrive testo nella txt

# Creo checkbox
def chkMaggiorenne_checkedChange():
    if v_chkMaggiorenne.get():
        messagebox.showinfo("chkMaggiorenne", "Maggiorenne")
    else:
        messagebox.showinfo("chkMaggiorenne", "Minorenne")

v_chkMaggiorenne = tk.BooleanVar() #Variabile di controllo per impostare/leggere se
è checked
v_chkMaggiorenne.set(False) #imposta chkMaggiorenne a non checked

chkMaggiorenne = tk.Checkbutton(frm, text="Maggiorenne", variable=v_chkMaggiorenne,
command=chkMaggiorenne_checkedChange)
chkMaggiorenne.place(x=100,y=50)

#Creazione radiobutton
def rdb_checkedChanged():#evento checked
    messagebox.showinfo("Radiobutton", f"Hai selezionato: {v_rdb.get()}")

v_rdb = tk.StringVar() #Variabile di controllo per impostare/leggere se è checked
v_rdb.set("s") #imposta rdbStudente a checked

rdbStudente = tk.Radiobutton(frm, text="Studente", variable=v_rdb, value="s",
command=rdb_checkedChanged)
rdbStudente.place(x=100,y=70)
rdbDocente = tk.Radiobutton(frm, text="Docente", variable=v_rdb, value="d",
command=rdb_checkedChanged)
rdbDocente.place(x=100,y=90)
```

```
#creazione combobox
cmbSpec = ttk.Combobox(frm, values=["Informatica", "Meccanica", "Elettrotecnica"])
cmbSpec.place(x=100,y=115)
cmbSpec.current(0) #cmbSpec.set("Informatica")
# Creo button
def btnInvia_on_click():
    ris = f"Ciao {txtNome.get()}, {v_chkMaggiorenne.get()}, {v_rdb.get()},
{cmbSpec.get()}-{cmbSpec.current()}"
    messagebox.showinfo("Titolo",ris)

btnInvia = tk.Button(frm, text="Invia", command=btnInvia_on_click)
btnInvia.place(x=170,y=150)

# Avvio del ciclo principale
frm.mainloop()
```

1. Esempio uso .pack()

Con `pack()` i controlli vengono organizzati in blocchi che vengono posizionati nel form mediante l'utilizzo di una serie di parametri (opzionali) `side`, `expand`, `fill`, `ipadx`, `ipady`, `padx`, `pady`, `anchor`

Side determina la direzione dei controlli

- `'top'`: posiziona i controlli dall'alto verso il basso verticalmente.
- `'bottom'`: posiziona i controlli dal basso verso l'alto verticalmente.
- `'left'`: posiziona i controlli da sinistra verso destra orizzontalmente.
- `'right'`: posiziona i controlli da destra verso sinistra orizzontalmente.

Expand determina lo spazio che deve occupare il controllo rispetto agli altri oggetti

Fill determina come il controllo deve riempire lo spazio a sua disposizione.

- `none`: occupa soltanto lo spazio necessario al suo contenuto.
- `x`: si espande orizzontalmente.
- `y`: si espande verticalmente.
- `both`: si espande in entrambe le direzioni.

ipadx, ipady, padx, pady determina il `padding` interno ed esterno intorno al controllo

Anchor ancora i controlli ai bordi del form

```
import tkinter as tk

frm = tk.Tk()
frm.title('Tkinter Pack Layout')
frm.geometry('600x400')

lbl1 = tk.Label(frm, text='Tkinter',bg='red',fg='white')
lbl2 = tk.Label(frm,text='Pack',bg='green', fg='white')
lbl3 = tk.Label(frm, text='Layout',bg='blue', fg='white')
lbl4 = tk.Label(frm, text='Demo',bg='purple', fg='white')
```

```

lbl1.pack(side=tk.TOP, expand=False, fill='x')
lbl2.pack(side=tk.TOP, expand=True, fill='y')
lbl3.pack(side=tk.TOP, expand=True, fill='none', anchor='e')
lbl4.pack(side=tk.TOP, expand=True, fill='both')

frm.mainloop()

```

2. Esempio uso .grid()

Con `grid()` i controlli vengono posizionati in una griglia (matrice) di righe e colonne mediante due metodi `columnconfigure()`, `rowconfigure()` ed una serie di parametri (opzionali) `column`, `row`, `colspan`, `rowspan`, `sticky`, `padx`, `pady`, `ipadx`, `ipady`

Columnconfigure() e rowconfigure() `index` colonna/riga che si vuole configurare, `weight` larghezza/altezza rispetto alle altre

Column e row indicano la cella in cui inserire il controllo

Rowspan e colspan indicano il numero di righe/colonne adiacenti che devono essere unite in un'unica cella

Sticky indica il lati della cella a cui far aderire un controllo più grande della cella

ipadx, ipady, padx, pady determina il `padding` interno ed esterno intorno al controllo

```

import tkinter as tk
from tkinter import ttk

# form
frm = tk.Tk()
frm.geometry("240x100")
frm.title('Login')
frm.resizable(False, False) #form non ridimensionabile

# imposto larghezza colonne
frm.columnconfigure(0, weight=1)
frm.columnconfigure(1, weight=3)

# username
lblUsername = ttk.Label(frm, text="Username:")
lblUsername.grid(column=0, row=0, sticky=tk.W, padx=5, pady=5)

txtUsername = ttk.Entry(frm)
txtUsername.grid(column=1, row=0, sticky=tk.E, padx=5, pady=5)

# password
lblPwd = ttk.Label(frm, text="Password:")
lblPwd.grid(column=0, row=1, sticky=tk.W, padx=5, pady=5)

txtPwd = ttk.Entry(frm, show="*")

```

```
txtPwd.grid(column=1, row=1, sticky=tk.E, padx=5, pady=5)

# login
btnLogin = ttk.Button(frm, text="Login")
btnLogin.grid(column=1, row=3, sticky=tk.E, padx=5, pady=5)

frm.mainloop()
```

3. Esempio uso .place()

Con `place()` i controlli vengono posizionati utilizzando la loro posizione assoluta o relativa `x`, `y`, `relx`, `rely`.

È possibile specificare la dimensione `width`, `height`, `relwidth`, `relheight` ed un punto di ancoraggio `anchor` che indica quale parte del controllo deve essere posizionata alle coordinate indicate

```
import tkinter as tk

frm = tk.Tk()
frm.title('Tkinter Place Geometry Manager')
frm.geometry("600x400")

#posizione assoluta
lblAssoluta = tk.Label(frm, text="Posizione assoluta",bg='red',fg='white')
lblAssoluta.place(x=20, y=10, width=120, height=60)

#posizione relativa
lblRelativa = tk.Label(frm, text="Posizione relativa",bg='blue',fg='white')
lblRelativa.place(relx=0.77, rely=0, relwidth=.2, relheight=.1) #valori compresi
tra 0.0 e 1.0

#anchor
lblAnchor = tk.Label(frm, text="Posizione anchor",bg='green',fg='white')
#posiziono il centro della label nel centro del form
lblAnchor.place(relx=0.5, rely=0.5, anchor='center', width=120, height=60)

frm.mainloop()
```

4. Esempio uso MessageBox

```
import tkinter as tk
from tkinter import messagebox

def mostra_messaggio():
    messagebox.showinfo("Titolo", "Questo è un messaggio informativo.")

def mostra_errore():
    messagebox.showerror("Errore", "Si è verificato un errore!")

def mostra_conferma():
    risposta = messagebox.askyesno("Conferma", "Vuoi continuare?")
    if risposta:
```

```
        print("L'utente ha scelto di continuare.")
    else:
        print("L'utente ha scelto di annullare.")

frm = tk.Tk()
frm.title("Esempio di Messagebox")
frm.geometry("300x200")

btn_info = tk.Button(frm, text="Mostra Messaggio", command=mostra_messaggio)
btn_info.pack(pady=10)

btn_error = tk.Button(frm, text="Mostra Errore", command=mostra_errore)
btn_error.pack(pady=10)

btn_conferma = tk.Button(frm, text="Mostra Conferma", command=mostra_conferma)
btn_conferma.pack(pady=10)
# Avvio del ciclo principale
frm.mainloop()
```